

CSE 3204
Formal Language, Automata and Computability



Lecture 5
**Application of NFA and DFA, and Regular
Expression and Languages**

An Application: Text Search

- Given a set of keywords, we want to find the occurrence of these words in a document.
- NFA for text search:
 1. There is a start state with a transition to itself on every input symbol, e.g. every printable ASCII character if we are examining text. Intuitively, the start state represents a “guess” that we have not yet begun to see one of the keywords, even if we have seen some letters of one of these words.
 2. For each keyword $a_1a_2 \cdots a_k$, there are k states, say $q_1, q_2, \dots, q_k$. There is a transition from the start state to q_1 on symbol a_1 , a transition from q_1 to q_2 on symbol a_2 , and so on. The state q_k is an accepting state and indicates that the keyword $a_1a_2 \cdots a_k$ has been found.

NFA for Text Search

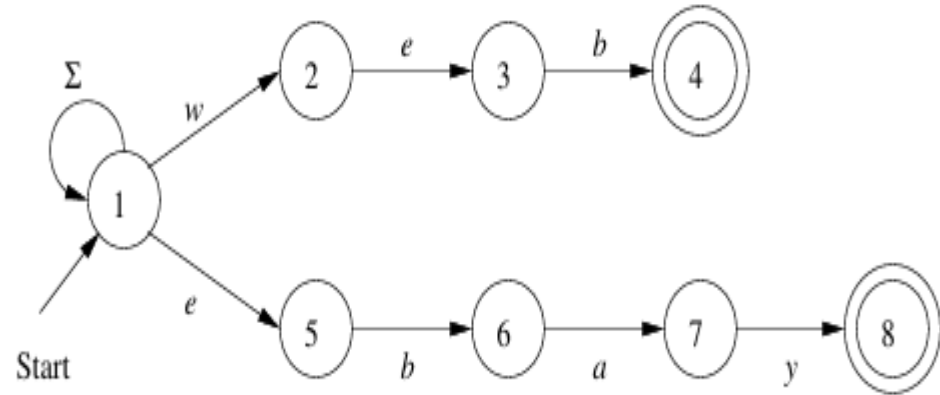


Figure 2.16: An NFA that searches for the words **w**eb and **e**bay

If subset construction is applied to build DFA, the number of states will not exceed the number of states in corresponding NFA.

DFA for Text Search (using subset construction)

- a) If q_0 is the start state of the NFA, then $\{q_0\}$ is one of the states of the DFA.
- b) Suppose p is one of the NFA states, and it is reached from the start state along a path whose symbols are $a_1 a_2 \cdots a_m$. Then one of the DFA states is the set of NFA states consisting of:
1. q_0 .
 2. p .
 3. Every other state of the NFA that is reachable from q_0 by following a path whose labels are a suffix of $a_1 a_2 \cdots a_m$; that is, any sequence of symbols of the form $a_j a_{j+1} \cdots a_m$.

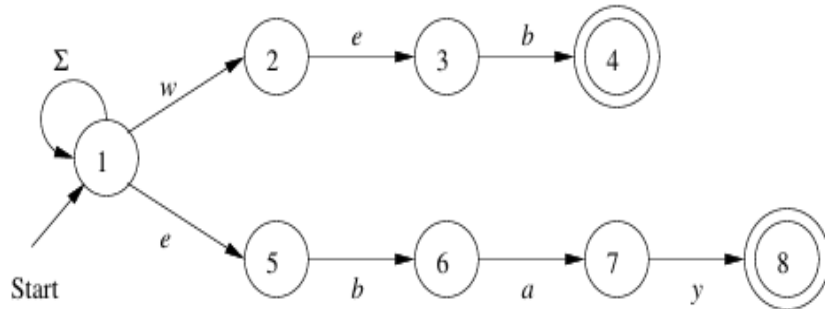


Figure 2.16: An NFA that searches for the words **web** and **ebay**

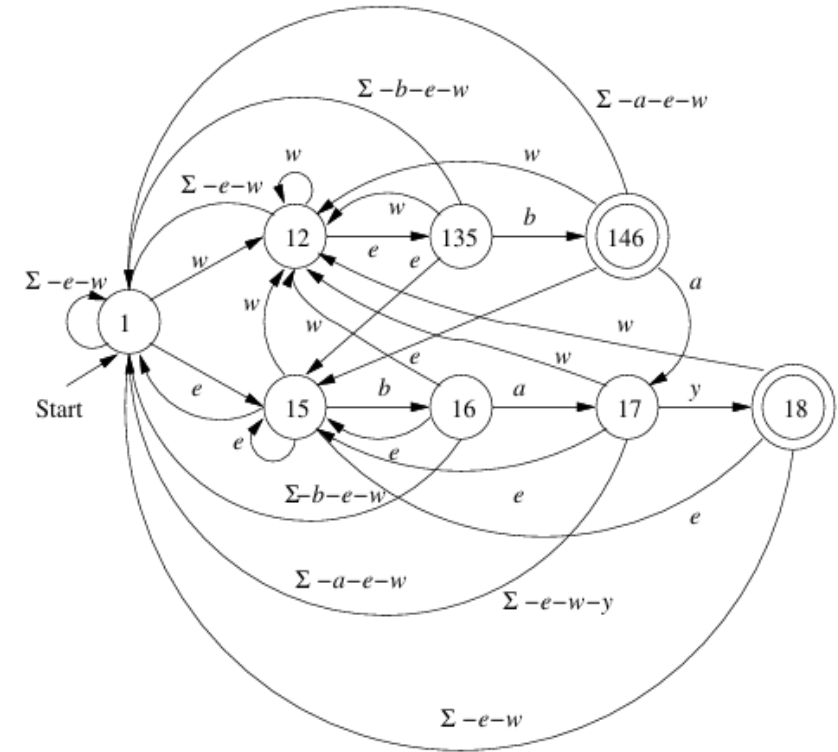


Figure 2.17: Conversion of the NFA from Fig. 2.16 to a DFA

Regular Expression

- Another type of language defining notation.
- Closely related to NFA.
- Instead of machine-like description of languages, it is an algebraic description.
- regular expressions can define exactly the same languages that the various forms of automata describe: regular language.
- serve as the input language for many systems that process strings

Operators of Regular Expression

The *union* of two languages L and M , denoted $L \cup M$, is the set of strings that are in either L or M , or both. For example, if $L = \{001, 10, 111\}$ and $M = \{\epsilon, 001\}$, then $L \cup M = \{\epsilon, 10, 001, 111\}$.

The *concatenation* of languages L and M is the set of strings that can be formed by taking any string in L and concatenating it with any string in M . Recall Section 1.5.2, where we defined the concatenation of a

$\bar{L} = \{001, 10, 111\}$ and $M = \{\epsilon, 001\}$, then $L.M$, or just LM , is $\{001, 10, 111, 001001, 10001, 111001\}$

The *closure* (or *star*, or *Kleene closure*)¹ of a language L is denoted L^* and represents the set of those strings that can be formed by taking any number of strings from L , possibly with repetitions (i.e., the same string may be selected more than once) and concatenating all of them. For instance, if $L = \{0, 1\}$, then L^* is all strings of 0's and 1's. If $L = \{0, 11\}$, then L^* consists of those strings of 0's and 1's such that the 1's come in pairs, e.g., 011, 11110, and ϵ , but not 01011 or 101. More formally, L^* is the infinite union $\cup_{i \geq 0} L^i$, where $L^0 = \{\epsilon\}$, $L^1 = L$, and L^i , for $i > 1$ is $LL \cdots L$ (the concatenation of i copies of L).

Operators of Regular Expression

Example 3.1: Since the idea of the closure of a language is somewhat tricky, let us study a few examples. First, let $L = \{0, 11\}$. $L^0 = \{\epsilon\}$, independent of what language L is; the 0th power represents the selection of zero strings from L . $L^1 = L$, which represents the choice of one string from L . Thus, the first two terms in the expansion of L^* give us $\{\epsilon, 0, 11\}$.

Next, consider L^2 . We pick two strings from L , with repetitions allowed, so there are four choices. These four selections give us $L^2 = \{00, 011, 110, 1111\}$. Similarly, L^3 is the set of strings that may be formed by making three choices of the two strings in L and gives us

$$\{000, 0011, 0110, 1100, 01111, 11011, 11110, 111111\}$$

To compute L^* , we must compute L^i for each i , and take the union of all these languages. L^i has 2^i members. Although each L^i is finite, the union of the infinite number of terms L^i is generally an infinite language, as it is in our example.

Now, let L be the set of all strings of 0's. Note that L is infinite, unlike our previous example, which is a finite language. However, it is not hard to discover what L^* is. $L^0 = \{\epsilon\}$, as always. $L^1 = L$. L^2 is the set of strings that can be formed by taking one string of 0's and concatenating it with another string of 0's. The result is still a string of 0's. In fact, every string of 0's can be written as the concatenation of two strings of 0's (don't forget that ϵ is a "string of 0's"; this string can always be one of the two strings that we concatenate). Thus, $L^2 = L$. Likewise, $L^3 = L$, and so on. Thus, the infinite union $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$ is L in the particular case that the language L is the set of all strings of 0's.

For a final example, $\emptyset^* = \{\epsilon\}$. Note that $\emptyset^0 = \{\epsilon\}$, while $\emptyset^i$, for any $i \geq 1$, is empty, since we can't select any strings from the empty set. In fact, $\emptyset$ is one of only two languages whose closure is *not* infinite. $\square$

Building Regular Expression

BASIS: The basis consists of three parts:

1. The constants ϵ and $\emptyset$ are regular expressions, denoting the languages $\{\epsilon\}$ and $\emptyset$, respectively. That is, $L(\epsilon) = \{\epsilon\}$, and $L(\emptyset) = \emptyset$.
2. If a is any symbol, then **a** is a regular expression. This expression denotes the language $\{a\}$. That is, $L(\mathbf{a}) = \{a\}$. Note that we use boldface font to denote an expression corresponding to a symbol. The correspondence, e.g. that **a** refers to a , should be obvious.
3. A variable, usually capitalized and italic such as L , is a variable, representing any language.

INDUCTION: There are four parts to the inductive step, one for each of the three operators and one for the introduction of parentheses.

1. If E and F are regular expressions, then $E + F$ is a regular expression denoting the union of $L(E)$ and $L(F)$. That is, $L(E + F) = L(E) \cup L(F)$.
2. If E and F are regular expressions, then EF is a regular expression denoting the concatenation of $L(E)$ and $L(F)$. That is, $L(EF) = L(E)L(F)$.

Building Regular Expression

3. If E is a regular expression, then E^* is a regular expression, denoting the closure of $L(E)$. That is, $L(E^*) = (L(E))^*$.
4. If E is a regular expression, then (E) , a parenthesized E , is also a regular expression, denoting the same language as E . Formally; $L((E)) = L(E)$.

Example- Building Regular Expression

- Construct a regular expression for the set of strings that consists of alternating 0's and 1's.

Basis step:

0 and 1 are regular expressions denoting language {0} and {1}.

Induction step:

Concatenate 0 and 1 to get 01 regular expression representing language {01}.

For all expressions consist of zero or more occurrence of 01, the regular expression is $(01)^*$

However, $L((01)^*)$ is not the exact L representing all strings consists of alternating 0's and 1's.

Final regular expression $(01)^* + (10)^* + 0(10)^* + 1(01)^*$

Example- Building Regular Expression

Alternative Representation: $(\epsilon + 1)(01)^*(\epsilon + 0)$

Precedence of Regular Expression Operators

1. The star operator is of highest precedence. That is, it applies only to the smallest sequence of symbols to its left that is a well-formed regular expression.
2. Next in precedence comes the concatenation or “dot” operator.
3. Finally, all unions (+ operators) are grouped with their operands.

Precedence of Regular Expression Operators

$01^* + 1$ is grouped $(0(1^*)) + 1$

$(01)^* + 1$

$0(1^* + 1)$

- Section 2.4 of Chapter 2.
- Section 3.1 of Chapter 3.